

sp(eye) – SEGP Group 2 Report

Alexander Chow, Caleb Kan, Jeffrey Cheung, Kee Meng Tan, Royce Chan, Ryan Ghoussainy

{akc123, cmk123, jhc23, kmt123, rlc23, rg1523}@ic.ac.uk

March 6, 2026

1 Introduction

Speed reading tools have a blind spot: they measure how fast you read, but not whether you understood any of it. Most present text at a fixed pace with no comprehension feedback, so users cannot tell if they are genuinely improving or just learning to skim. The real challenge is building a system that can observe and adapt to how quickly someone reads, and assess their understanding in real time.

sp(eye) is the web platform we built to address this, using a React and TypeScript frontend, and Supabase for the backend. It offers three reading modes: Standard (with a configurable WPM (words per minute)), Adaptive (webcam-based eye-tracking using WebGazer.js [1]), and RSVP (mimicking mobile short-form content). Texts come with LLM-generated comprehension quizzes immediately after each session [2] and a dashboard tracks users' speed and accuracy over time. Our key contributions include:

- **Webcam-driven adaptive reading** that adjusts pace to match the user's gaze in real time, requiring no additional hardware.
- **Two-stage AI pipeline** for content moderation, title generation, quiz generation and validation, genre classification, and summarisation.
- **Competitive PvP multiplayer** with Elo ranking, real-time synchronisation, and a ranked leaderboard.
- **Text Library and admin moderation** with public and personal libraries, filtering and search, and a role-based admin dashboard for approving and editing texts with access to global analytics.
- **Scoring and analytics** combining a chance-corrected formula that balances speed against comprehension with a personal activity dashboard for tracking improvement over time.

2 Overview of what was produced

The project was built largely from scratch. The one exception is WebGazer.js [1], whose source code was embedded directly into the repository. All other code, database schemas, edge functions, and infrastructure configuration are original contributions from the team.

2.1 System Architecture

The platform follows a serverless architecture with three main components: a React single-page application, a Supabase Backend-as-a-Service layer [3], and two external services (Groq for AI inference [4] and Upstash Redis for quiz leaderboards [5]). Figure 1 summarises the relationships.

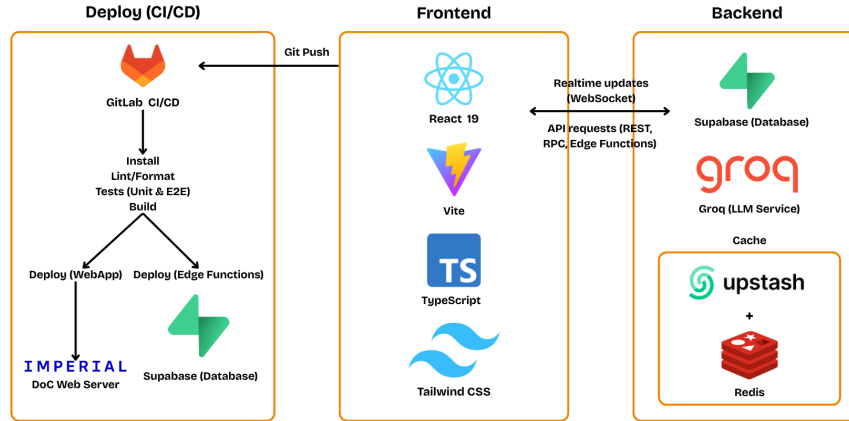


Figure 1: High-level system architecture of sp(eye).

2.2 Frontend

The frontend is a Progressive Web App served from a single HTML entry point. React Router handles client-side routing across pages including Home (standard reading), Adaptive, RSVP, Library, Activity dashboard, PvP lobby, PvP game, Admin, Settings, Profile, Notifications, and supporting authentication/legal pages. Five React context providers wrap the application tree: **AuthProvider** (Supabase session), **ThemeProvider** (multiple colour schemes), **NetworkStatusProvider** (online detection), **ReadingPreferencesProvider** (WPM, mode), and **NotificationsProvider**. Mobile users are automatically redirected from the desktop reading modes to RSVP.



Figure 2: User Webpage Flow and Website Architecture.

2.2.1 Reading Modes

Standard mode renders the full text at a configurable WPM rate, highlighting the current word. Adaptive mode displays one line at a time while WebGazer.js streams gaze coordinates at up to 30 Hz via TensorFlow.js [6] and MediaPipe [7]. We added custom hooks to apply gaze smoothing and return-sweep detection to advance the text automatically to ensure a seamless experience. Calibration drift is detected and flagged. RSVP mode presents a few words at a time at the centre of the screen, optimised for narrow screens [8]. A Summary variant is available for non-fiction texts across all three modes, displaying a LLM-generated summary instead of the full text.

2.2.2 Text Library

The Library is split into two tabs: Private (the user’s own uploads) and Public (admin-approved texts available to all users). Users can upload plain or sectional texts, which are then processed asynchronously through the AI pipeline. Real-time status updates appear as the text moves through processing. Both tabs support filtering by genre (fiction/non-fiction), complexity range, and free-text search, with sorting by date or complexity. Each text card shows the user’s best score and last-read date. Users can edit or delete their own texts, and non-fiction texts offer a “Read Summary” option that opens the LLM-generated summary in the active reading mode.

2.2.3 Admin Interface

Administrators access a dedicated dashboard protected by role-based access control (ROLE_ADMIN). It exposes controls for approving, editing, and deleting texts, retrying failed processing jobs, promoting users, and viewing platform-wide statistics via server-side SQL functions.

2.3 Backend

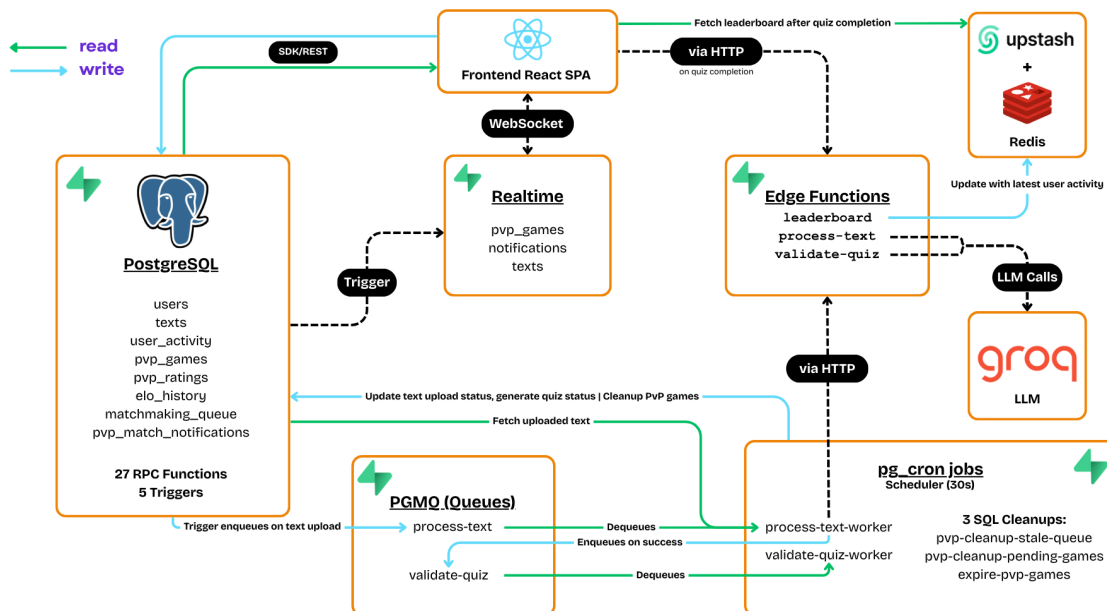


Figure 3: High-level backend architecture of sp(eye).

2.3.1 AI Text Processing Pipeline

Uploaded texts are processed asynchronously through a queue-driven, two-stage pipeline. When a user uploads a text, it is placed onto a PostgreSQL message queue (pgmq). A `process-text-worker`, triggered periodically by `pg_cron`, reads one job at a time from the queue, fetches the text from the database, and calls the `process-text` edge function. This first stage calls Groq using `openai/gpt-oss-120b` with structured JSON schema output to screen the text for Terms of Service violations, generate between one and five sets of five-to-seven four-option comprehension questions [9], classify the text as fiction or non-fiction, infer a title if absent, and produce a non-fiction summary. Rate limiting (20 requests per minute per IP) is enforced in-process. On success, the worker writes the results back to the database and enqueues a second job onto the `validate_quiz` queue. A `validate-quiz-worker`, also `pg_cron`-triggered, then calls the `validate-quiz` edge function, which uses a different model (`llama-3.3-70b-versatile` [10]) to independently verify that all generated questions are genuinely answerable from the source text. We use a separate model for validation since a generator should not grade its own output, reducing the risk of self-consistent but unfaithful questions [11]. If validation fails, the text is flagged for admin review with a retry option. Both workers use visibility timeouts to prevent duplicate processing and delete queue messages on completion or failure to avoid infinite retry loops. The entire pipeline runs without blocking the client, and texts up to 15,000 characters are supported.

2.3.2 Overall Scoring

Each reading session produces an Overall Score that balances speed against verified comprehension:

$$\text{Overall Score} = \left[\text{WPM}^{0.7} \times \left(\frac{A - 0.25}{0.75} \right)^{1.5} \times 10 \right]$$

where A is the raw quiz accuracy (0–1). The chance-correction floor at 25% neutralises random guessing on four-option questions [12]. The super-linear accuracy exponent (1.5) penalises poor comprehension more than raw speed [13], while the sub-linear speed exponent (0.7) reflects diminishing returns beyond a reader’s comfortable pace [14]. Per-text leaderboards are cached in Upstash Redis: after each quiz, a `leaderboard` edge function writes the user’s best score into a sorted set with a seven-day TTL. The frontend reads rankings directly from Redis, keeping leaderboard queries off the primary database. This helps reduce the query complexity of the large `user_activity` table.

2.3.3 Player-versus-Player Mode

PvP matches are coordinated through Supabase Realtime. The matchmaking queue pairs users by Elo proximity; once matched, both clients join a shared broadcast channel for the game session. Progress updates, quiz submissions, and forfeit signals are exchanged via broadcast events with acknowledgement enabled, while a heartbeat is sent every five seconds to track presence. Three `pg_cron` cleanup jobs run periodically to remove stale records from the matchmaking queue, mark abandoned pending games, and expire active PvP game sessions, preventing unbounded table growth. Both players read the same randomly selected text, then answer the same quiz within 20 seconds per question. Results are submitted atomically; forfeit and AFK detection trigger after 15s of inactivity (warning) and 30s (automatic forfeit). Elo is updated server-side using a standard K -factor model [15]. Ranks are structured across 18 tiers formed by combining six animals (Snail, Turtle, Rabbit, Sparrow, Chimp, Cheetah) with three levels (Baby, Young, Prime), so tiers progress from Baby Snail up to Prime Cheetah.

3 Evaluation

User testing – We conducted user testing through a survey with 12 users from technical and non-technical backgrounds. The participants were asked to complete scenario-based tasks (e.g. reading a text in different modes, uploading a text, etc.) and provide qualitative feedback on their experience and how the website met their user needs. We have rapidly addressed all of their feedback, some of the changes including adding tooltips for reading modes, sectional texts, as well as mobile support.

Product Robustness – We ensured device compatibility for both desktop and mobile and added an RSVP mode, which is more suitable for smaller screens. The website is installable as a Progressive Web App with offline support. For progress tracking, the scoring algorithm’s [chance-correction](#) floor prevents inflated scores, making sure that the Activity dashboard reflects genuine comprehension improvement over time alongside reading speed. The text library is protected by a two-stage moderation pipeline: both uploaded texts and their generated quizzes are first screened by an LLM for Terms of Service violations and quality, followed by a review and an approval by a human administrator. Finally, over 125 unit test suites (Vitest) and 13 end-to-end test suites (Playwright) run across Chromium, Firefox, and WebKit, verifying that every user receives a seamless experience regardless of their browser.

Strengths – We have quantitative metrics in place to measure the platform’s effectiveness. Each reading session records words per minute and quiz accuracy, all visible on the Activity dashboard over configurable timeframes (7-day, 30-day, and all-time), allowing users to track personal trends in both speed and comprehension. On the admin side, WPM distribution charts and global quiz accuracy averages provide a platform-wide view of aggregated analytics. These metrics ensure we stay aligned with our core goal: speed reading should not come at the cost of comprehension. However, drawing statistically meaningful conclusions requires a larger user base than we currently have.

What’s missing? – All original requirements from the project brief were fully met. Beyond the specification, we implemented PvP multiplayer with Elo ranking, a mobile-optimised RSVP mode, sectional reading for longer texts, a notifications system, and offline caching with automatic sync. Looking ahead, a limitation of our platform is the quality of the LLM outputs. The model occasionally produces quiz questions which are obviously wrong, making it easier for the reader to eliminate answers and inflate their score. Solving this would require careful prompt engineering and stricter validation steps in our pipeline. On the scalability side, the serverless architecture (Supabase Edge Functions, Groq inference, Upstash Redis) already scales horizontally with demand. The main bottleneck at higher user counts would be rate limits on text processing or text fetching, which could be addressed by adding rate-limit backoff to the existing processing queue or upgrading to premium rate limit plans.

4 Declarations

Use of Generative AI

Claude Code [16] was used during development to suggest code completions, generate boilerplate, and assist with implementation tasks. All AI-generated suggestions had to be reviewed, and understood by team members before being committed to the repository. The submitted work is our own.

Ethical Considerations

The principal ethical risk is webcam use for eye-tracking. All gaze data is processed client-side via WebGazer.js and TensorFlow.js; no video frames or coordinates are transmitted to the server or stored persistently. Users must give explicit consent before entering Adaptive mode. A Terms of Service and Privacy

Policy are accessible from every page. User testing was conducted with informed consent and no sensitive personal data was collected. Uploaded texts pose a content-moderation risk. This is handled by a two-stage pipeline involving an LLM check and an admin approval, as mentioned above.

Sustainability

The platform is entirely serverless, scaling to zero when idle. The PWA offline cache and 90-day client-side caching of TensorFlow/MediaPipe models minimise redundant downloads. AI inference is rate-limited and triggered only on text upload, not during reading sessions. Groq's LPU™ Inference Engine consumes as little as one-tenth the power of a comparable GPU per chip and is powered by renewable energy [17].

Availability of Data and Materials

The full source code for the platform is available in our GitLab repository at <https://gitlab.doc.ic.ac.uk/g256002102/segp>. A live production deployment of the web application is available at <https://speye.doc.ic.ac.uk/> and can be accessed with any modern browser.

References

- [1] Papoutsaki A, Sangkloy P, Laskey J, Daskalova N, Huang J, Hays J. WebGazer: scalable webcam eye tracking using user interactions. In: *Proc 25th Int Jt Conf Artif Intell (IJCAI)*; 2016.
- [2] Peterson LR, Peterson MJ. Short-term retention of individual verbal items. *J Exp Psychol.* 1959;58(3):193–198.
- [3] Supabase, Inc. Supabase documentation: database, auth, edge functions, realtime [Internet]. 2024.
- [4] Groq, Inc. Groq API documentation [Internet]. 2024.
- [5] Upstash, Inc. Upstash Redis documentation [Internet]. 2024.
- [6] TensorFlow Authors. TensorFlow.js: machine learning for the web and Node.js [Internet]. 2024.
- [7] Google. MediaPipe face landmarker [Internet]. 2024.
- [8] Fine EM, Peli E. Benefits of rapid serial visual presentation (RSVP) over scrolled text vary with letter size. *Optom Vis Sci.* 1998;75(3):191–196.
- [9] Szpunar KK, Khan NY, Schacter DL. Interpolated memory tests reduce mind wandering and improve learning of online lectures. *Proc Natl Acad Sci U S A.* 2013;110(16):6313–6317.
- [10] Meta AI. Llama 3.3 70B versatile [large language model]. Accessed via Groq inference API; 2024.
- [11] Zheng Y, Yan H, Qi Q, Song K, Xu H, Gui L, et al. Multi-agent LLM survey: advances, applications, and open problems. *arXiv preprint arXiv:2508.06709*. 2025.
- [12] Frary RB. Formula scoring of multiple-choice tests (correction for guessing). *Educ Meas Issues Pract.* 1988;7(2):33–38.
- [13] Liesefeld HR, Janczyk M. Combining speed and accuracy to control for speed–accuracy trade-offs(?). *Behav Res Methods.* 2019;51(1):40–60.
- [14] Rayner K, Schotter ER, Masson MEJ, Potter MC, Treiman R. So much to read, so little time: how do we read, and can speed reading help? *Psychol Sci Public Interest.* 2016;17(1):4–34.
- [15] Elo AE. The rating of chessplayers, past and present. Arco Publishing; 1978.
- [16] Anthropic. Claude Code [Internet]. 2025. Available from: <https://docs.anthropic.com/en/docs/claude-code>
- [17] Groq, Inc. Groq® & Earth Wind Power build energy-efficient AI center in Norway [press release]. 2024 Apr 22. <https://groq.com/groq-earth-wind-power-build-energy-efficient-ai-center-in-norway/>